Extracción de texto desde pdf/imágenes

Existe una biblioteca llamada **pypdf** que se puede utilizar para lo que queremos. Este paquete también se puede utilizar para generar, descifrar y fusionar archivos PDF.

Otra biblioteca para extraer texto recomendada es **PyMuPDF**.

Es una biblioteca que admite archivos XPS, PDF, CBR y CBZ.

Se pueden ver los pasos de código en el siguiente enlace:

https://www.geeksforgeeks.org/python/extract-text-from-pdf-file-using-python/

Siguiendo hablando de PyMuPDF, sirve muy bien para archivos PDF de texto nativo, pero requiere un desarrollo adicional para la integración de OCR, detección de tablas, manejo de formularios y recuperación de errores.

Nutrient API es una API basada en la nube que incluye OCR(archivos escaneados), análisis de diseño y actualizaciones automáticas. Maneja documentos nativos y escaneados sin requerir un trabajo de integración personalizada.

PyMuPDF puede extraer texto básico sin formato hasta datos detallados basados en coordenadas

Métodos de Extracción de texto:

PyMuPDF `page.get_text()` admite varios modos de extracción:

- `"texto"` Texto sin formato en orden de lectura(el más rápido para la extracción básica)
- `"bloques"` / `"palabras"` Listas de bloques o palabras con posiciones.
- `"dict"` Datos estructurados con fuentes, coordenadas e información de diseño.
- `"json"` Igual que `dict`, pero en formato JSON.
- `"html"`, `"xml"`, `"rawdict"`, si `"rawjson"` tambien disponible

Usar `Texto` para extracción sencilla, usar `dict` para cuando necesitas coordenadas para tablas o formularios

Extraccion de texto básica

```
import pymupdf

def extract_text_pymupdf(pdf_path):
    doc = pymupdf.open(pdf_path)
    try:
        text = ""
        for page in doc:
            text += page.get_text("text") # plain text
        return text
    finally:
        doc.close()

# Extract text from a PDF.
result = extract_text_pymupdf("invoice.pdf")
print(result)
```

Para tablas y formularios necesitas coordenadas

```
import pymupdf

def extract_with_coordinates(pdf_path):
    """Extract text with position and font information for
layout analysis."""
    doc = pymupdf.open(pdf_path)
    try:
        results = []
        # Process each page in the document.
        for page_num, page in enumerate(doc):
            # Get structured data: blocks contain lines, li
nes contain spans.
            data = page.get_text("dict")  # Returns hierarc
hical text structure
```

```
            # Navigate the hierarchy: blocks > lines > span
s.
            for block in data["blocks"]:
                # Skip image blocks (they don't have "line
s" key).
                for line in block.get("lines", []):
                    # Each span is the smallest text unit w
ith consistent formatting.
                    for span in line["spans"]:
                        results.append({
                            "page": page_num,
                            "text": span["text"],
                            "bbox": span["bbox"],  # (x0,y
0,x1,y1) coordinates in points
                            "font": span["font"]   # Font n
ame for styling analysis
                        })
        return results
    finally:
        # Always close the document to free memory.
        doc.close()

# Usage example: Extract positioned text for table detectio
n.
result = extract_with_coordinates("invoice.pdf")
print(result[:5])  # Preview first 5 text spans with positi
ons
```

Extracción de mesas(tablas bordeadas)

```
import pymupdf

def extract_tables_pymupdf(pdf_path):
    doc = pymupdf.open(pdf_path)
    try:
```

```
        all_tables = []
        for page in doc:
            # Returns a `TableFinder` object.
            table_finder = page.find_tables() # `TableFinde
r`
            for table in table_finder.tables: # Sequence of
`Table` objects
                all_tables.append(table.extract())  # Each
is a list of rows.
        return all_tables
    finally:
        doc.close()

# Usage
result = extract_tables_pymupdf("invoice.pdf")
for i, table in enumerate(result[:5]):  # First 5 tables.
    print(f"--- Table {i} ---")
    for row in table:
        print(row)
```

Manejo de archivos PDF escaneados(respaldo de OCR)

Esta sección describe como agregar Tesseract OCR para contenido escaneado.

- Instalar requisitos previos de OCR:

```
# Python deps
pip instalar pytesseract Almohada

# Motor Tesseract (paquete del sistema)
#macOS
cerveza instalar teseracto
#ubuntu/debian
sudo apt-get instalar tesseract-ocr
# Ventanas
# 1) Instalar desde: https://github.com/UB-Mannheim/tessera
```

```
ct/wiki
# 2) Agregue el directorio de instalación (p. ej. C:\Archiv
os de programa\Tesseract-OCR) a PATH
```

- Utilice el extractor compatible con OCR:

```
import pymupdf
import pytesseract
from PIL import Image
import io

def is_scanned(page, threshold=40) -> bool:
    """Check if page has little text (likely scanned/image-
only)."""
    return len(page.get_text("text").strip()) < threshold

def extract_with_ocr(pdf_path: str, ocr_lang: str = "eng")
-> str:
    """Hybrid extraction: use native text when available, O
CR when needed."""
    doc = pymupdf.open(pdf_path)
    try:
        out = []
        for page in doc:
            if is_scanned(page):
                # Page has minimal text — likely scanned, u
se OCR.
                pix = page.get_pixmap(dpi=300)  # Render at
300 DPI for good OCR quality.
                img = Image.open(io.BytesIO(pix.tobytes("pn
g")))  # Convert to PIL Image
                # Run Tesseract OCR with specified languag
e.
                out.append(pytesseract.image_to_string(img,
lang=ocr_lang))
            else:
                # Page has native text — extract directly
(much faster).
```

```
                out.append(page.get_text("text"))
        return "".join(out)
    finally:
        doc.close()

# Example usage with error handling.
if __name__ == "__main__":
    try:
        result = extract_with_ocr("invoice.pdf")
        print(f"Extracted {len(result)} characters")
        print(result[:500])  # Preview first 500 character
s.
    except Exception as e:
        print(f"Extraction failed: {e}")
```

> Para documentos mas escaneados, primero preprocesarlos con OpenCV utilizando técnicas de binarización, desescarchado y eliminación de ruido.

Mejoras prácticas de desempeño

- Abierto una vez llamada → `pymupdf.open()` una vez por documento. No vuelvas a abrir para cada página.
- Preferir `texto` modo Uso → `page.get_text("texto")`, a menos que necesites coordenadas o datos de fuente.
- Omitir renderizado innecesario → `page.get_pixmap()` es lento, así que utilicelo solo para OCR.
- Manejar archivos no válidos Uso→ `intentar/excepto` para archivos PDF corruptos.
- Paralelizar lotes→ `Documentos` los objetos son independientes, por lo que utilice el multiprocesamiento para trabajos masivos.

Api del procesador Nutrient DWS para Python

La API del procesador Nutrient DWS es una API de procesamiento de PDF basada en la nube con OCR incorporado y extracción de datos. Una llamada API maneja:

- Impulsado por IA y aprendizaje automático OCR que procesa escaneos, escritura a mano y tipos de archivos mixtos de mala calidad
- Orden de lectura inteligente que mantiene el flujo de texto lógico en diseños complejos o multicolumnos
- Comprensión adaptativa del diseño que reconoce encabezados, párrafos, listas y secciones de documentos
- Detección de pares clave-valor para formularios, facturas y otros documentos estructurados
- Análisis consciente del diseño que preserva las relaciones espaciales entre texto, imágenes y anotaciones

> Para mas info sobre la api ver la página:https://www.nutrient.io/blog/extract-text-from-pdf-pymupdf/#step-1-sign-up-and-get-your-api-key

Tutorial de como extraer texto con PyMuPDF:
https://thepythoncode.com/article/extract-text-from-pdf-in-python

Documentación Oficial acerca de PyMuPDF:

https://pymupdf.io

PyTesseract

es un Wrapper de python para el motor OCR Tesseract de Google, diseñado para reconocer y extraer texto de imagenes. Convierte formatos como JPEG, PNG, GIF,TIFF en texto legible por máquina. Es una herramienta popular de código abierto para digitalizar documentos y procesar imagenes.
video Explicativo: https://youtu.be/OutxRdkNOK4?si=IveK3IioKfdX1PVK

OCR(Optical Character Recognition)

Para aplicar OCR a un código de extracción de texto de PDF, el enfoque cambia según si el PDF tiene texto digital (capas de texto) o si es una imagen (escaneado). **Si el PDF es una imagen, necesitas convertir las páginas en imágenes y luego procesarlas con un motor de OCR como Tesseract**.

Aquí tienes el procedimiento estándar utilizando **Python**, que es el lenguaje más común para esta tarea:

1. Preparar el entorno

Primero, necesitas instalar las librerías necesarias. `pytesseract` actúa como interfaz para el motor Tesseract, y `pdf2image` convierte las páginas del PDF en imágenes que el OCR pueda leer.

2. Convertir PDF a imágenes

Dado que los motores de OCR no "leen" archivos PDF directamente, el primer paso es renderizar cada página como una imagen (habitualmente en formato `.jpg` o `.png`).

python

```
from pdf2image import convert_from_path
import pytesseract

# 1. Convertir las páginas del PDF a una lista de objetos d
e imagen
paginas = convert_from_path('archivo_escaneado.pdf', dpi=30
0)
```

3. Aplicar el motor OCR

Una vez que tienes las imágenes, iteras sobre ellas para extraer el texto. Es recomendable pre-procesar la imagen (escala de grises, binarización) si el documento tiene mucho ruido, aunque Tesseract lo hace internamente de forma básica.

python

```
texto_completo = ""

for i, pagina in enumerate(paginas):
    # 2. Aplicar OCR a cada imagen de página
    # Puedes especificar el idioma con lang='spa'
    texto = pytesseract.image_to_string(pagina, lang='spa')

    texto_completo += f"--- Página {i+1} ---\n"
    texto_completo += texto + "\n"
```

```
print(texto_completo)
```

Usa el código con precaución.

4. Guardar o procesar el resultado

Finalmente, puedes volcar el contenido extraído en un archivo de texto simple o un JSON para su posterior análisis.

```
with open("resultado_ocr.txt", "w", encoding="utf-8") as f:
    f.write(texto_completo)
```

• **PyMuPDF (fitz)**: Excelente para extraer texto digital rápidamente y tiene integraciones nativas para llamar a Tesseract si detecta imágenes.

Lo que falta por ver:

Seguridad / lógica

- Checksum (hash MD5/SHA)
- Evitar duplicados

Persistencia

- MongoDB (NoSQL)

7. Búsquedas que podés hacer

Si necesitás más info, buscá esto:

- `python extract text pdf pymupdf`
- `fastapi upload file pdf python`
- `python pdf checksum hashlib`
- `pytesseract python example`
- `mongodb python fastapi beanie`